

Manager's Meeting: Next Step – Coding Phase

Objective of This Meeting

Explain to students:

How we move from diagrams to actual coding

What modules will be developed

How the team will work

What deliverables are expected

1) Recap Before Coding

Start your session with:

“We have already completed:”

Flowchart (Business process)

ER Diagram (Database design)

Activity Diagrams (System behavior)

Role identification (Admin, Counselor, Trainer, Student)

“Now we are entering the Coding Phase.”

2) How Coding Will Start (Step-by-Step Plan)

Step 1: Database Creation

Students must first create the database.

Tables to Create:

Users

Enquiries

Call_Records

Followups

Admissions

Courses

Students

Explain:

“Without a database, the software cannot function.”

Step 2: Backend Development

This is where the logic is written.

Students must implement:

Login authentication

Role-based access control

CRUD operations:

Create

Read

Update

Delete

Examples:

Add enquiry

Update follow-up

Convert admission

Step 3: Frontend Development

This is the user interface.

Pages to Create:

Login page

Dashboard (based on role)

Enquiry form

Call entry form

Follow-up page

Admission page

Step 4: Integration

Connect:

Frontend

Backend

Database

Explain:

“This is where the system becomes a fully working software.”

3) Module-wise Coding Breakdown

Module 1: Login System

Students must code:

Login form

Authentication logic

Role-based redirection

Output:

Admin dashboard

Counselor dashboard

Trainer dashboard

Student dashboard

Module 2: Enquiry Module

Students must code:

Add enquiry form

Save enquiry to database

Assign counselor

Module 3: Calling Module

Students must code:

Add call record

Store call status

Display call history

Module 4: Follow-up Module

Students must code:

Schedule follow-up

Display today's follow-ups

Update follow-up status

Module 5: Admission Module

Students must code:

Convert enquiry into admission

Store course and fee details

Module 6: Trainer and Student Module

Students must code:

Assign trainer to student

Display student details

Track attendance (basic level)

Module 7: Reports Module

Students must code:

Display total enquiries

Display total admissions

Show counselor performance

4) Team Structure

Divide students into roles:

Backend Developer

Frontend Developer

Database Designer

Tester

Explain:

“This is how real IT teams are structured in companies.”

5) Coding Standards

Students must follow:

Proper naming conventions

Modular coding structure

Code comments

Input validation

Avoid hardcoding values

6) Deliverables Expected from Students

Students must submit:

1. Working CRM Software

2. Source Code

3. Database Schema

4. All Diagrams

5. Screenshots of the system

6. Project Presentation

7) Suggested Timeline

Day 1–2: Database and Login

Day 3–4: Enquiry and Calling

Day 5–6: Follow-up and Admission

Day 7: Testing and UI improvement

8) Real-World Explanation

Tell students:

“This CRM is not just a project. This is exactly how real companies design and build software systems.”

9) Closing Statement

You can conclude with:

“Now we are moving from planning to execution.
Your diagrams were the blueprint, and coding is the construction.
By the end of this project, you will have built a real industry-level CRM system.”

-

Project Structure


```
crm_project/
|
|— app.py
|— models.py
|— database.db
|— templates/
|   |— login.html
|   |— dashboard.html
|   |— enquiry.html
|— static/
```

Database (SQL Sample)

You can first show students how tables are created.

SQL

```
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    email TEXT,
    password TEXT,
    role TEXT
);
```

```
CREATE TABLE enquiries (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    phone TEXT,
    course TEXT,
    status TEXT,
    counselor_id INTEGER
);
```

3) Main Backend Code (app.py)

This includes:

Login

Role-based dashboard

Add enquiry

Python

```
from flask import Flask, render_template, request, redirect, session
import sqlite3
```

```
app = Flask(__name__)
app.secret_key = "secret123"
```

```

# Database connection
def get_db():
    conn = sqlite3.connect("database.db")
    return conn

# ----- LOGIN -----
@app.route("/", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        email = request.form["email"]
        password = request.form["password"]

        conn = get_db()
        cursor = conn.cursor()
        cursor.execute("SELECT * FROM users WHERE email=? AND password=?", (email,
password))
        user = cursor.fetchone()

        if user:
            session["user_id"] = user[0]
            session["role"] = user[4]
            return redirect("/dashboard")
        else:
            return "Invalid Login"

    return render_template("login.html")

# ----- DASHBOARD -----
@app.route("/dashboard")
def dashboard():
    if "role" not in session:
        return redirect("/")

    role = session["role"]
    return render_template("dashboard.html", role=role)

# ----- ADD ENQUIRY -----
@app.route("/add_enquiry", methods=["GET", "POST"])
def add_enquiry():
    if request.method == "POST":
        name = request.form["name"]
        phone = request.form["phone"]
        course = request.form["course"]

```

```

    conn = get_db()
    cursor = conn.cursor()
    cursor.execute(
        "INSERT INTO enquiries (name, phone, course, status) VALUES (?, ?, ?, ?)",
        (name, phone, course, "New")
    )
    conn.commit()

    return "Enquiry Added Successfully"

return render_template("enquiry.html")

# ----- VIEW ENQUIRIES -----
@app.route("/enquiries")
def enquiries():
    conn = get_db()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM enquiries")
    data = cursor.fetchall()

    return str(data)

# ----- LOGOUT -----
@app.route("/logout")
def logout():
    session.clear()
    return redirect("/")

if __name__ == "__main__":
    app.run(debug=True)

```

4) Login Page (templates/login.html)

HTML

```

<h2>Login</h2>
<form method="POST">
    Email: <input type="text" name="email"><br>
    Password: <input type="password" name="password"><br>
    <button type="submit">Login</button>
</form>

```

5) Dashboard Page (templates/dashboard.html)

HTML

```

<h2>Dashboard</h2>

```

```

{% if role == 'admin' %}

```

```
<h3>Admin Panel</h3>
{% elif role == 'counselor' %}
<h3>Counselor Panel</h3>
{% endif %}

<a href="/add_enquiry">Add Enquiry</a><br>
<a href="/enquiries">View Enquiries</a><br>
<a href="/logout">Logout</a>
```

6) Enquiry Form (templates/enquiry.html)

HTML

```
<h2>Add Enquiry</h2>

<form method="POST">
  Name: <input type="text" name="name"><br>
  Phone: <input type="text" name="phone"><br>
  Course: <input type="text" name="course"><br>
  <button type="submit">Submit</button>
</form>
```

7) Sample Data Insert (Run Once)

SQL

```
INSERT INTO users (name, email, password, role)
VALUES ('Admin User', 'admin@gmail.com', '1234', 'admin');
```

```
INSERT INTO users (name, email, password, role)
VALUES ('Counselor User', 'counselor@gmail.com', '1234', 'counselor');
```

build:

Call Record Module

Follow-up Module

Admission Conversion

Proper UI (CSS / Bootstrap)